

한국사 암기어플 만들기 튜토리얼 (음성으로 답하는 역사 퀴즈)

0. 뭘 만드는 건지 먼저 알기

화면에 역사 사건 문제가 뜨면, 마이크에 대고 말로 답하는 앱을 만든다. 예를 들어 "1876년에 체결된 조약은?"이라는 문제가 뜨면 "강화도 조약이요"라고 말하고, 브라우저가 이 말을 텍스트로 바꿔서 정답인지 판정해준다.

핵심 원칙 하나 먼저 짚고 감

여기서 AI가 하는 일은 딱 하나, **음성을 텍스트로 바꾸는 것**까지다. "이 답이 맞았다/틀렸다"를 AI가 알아서 판단하는 게 아니다. 미리 정해둔 정답 키워드 목록이랑 인식된 텍스트를 비교해서 몇 개가 들어갔는지 세는 방식, 즉 **규칙 기반 채점**으로 처리한다. 리뷰 챗봇 만들 때 실시간으로 AI가 답을 만드는 게 아니라 미리 짚지어놓은 데이터에서 찾아오는 방식이었던 거랑 똑같은 원리다.

API 관련해서 하나 더

원래 프로젝트에서는 CLOVA Speech라는 네이버클라우드 유료 음성인식 API를 썼는데, 이 실습에서는 API 키 발급도 필요 없고 배포도 안 하는 게 원칙이라 **크롬 브라우저에 이미 내장된 무료 음성인식 기능 (Web Speech API)**으로 대체한다. 이걸 그냥 크롬에서 켜면 바로 되는 기능이라 회원가입도 결제도 필요 없다.

화면 상태 6개

상태	언제 뜨는지
대기	문제 표시, 마이크 누르기 전
녹음중	마이크 켜고 답변 받는 중
채점중	답변 다 말하고 채점하는 동안 (로딩)
정답	정답 키워드가 전부 들어감

상태	언제 뜨는지
부분정답	정답 키워드가 일부만 들어감
오답	정답 키워드가 하나도 없음

1. 사용할 툴

툴	용도
위키백과 / 교과서 자료	사건별 핵심 정보(연도, 인물, 사건명) 조사
크롬 개발자도구 (F12)	위키백과 등에서 텍스트 뽑을 때 (선택사항)
Gemini / ChatGPT	키워드 정리, 채점 기준 데이터셋 구성, 최종 코드 생성
구글 스프레드시트 / 엑셀	문제-정답 키워드 데이터 정리
메모장(또는 텍스트 편집기)	완성 코드를 .html 파일로 저장
크롬 브라우저	완성된 .html 파일 실행 (Web Speech API 쓰려면 크롬 권장)
Framer AI	화면 디자인 시안 참고용 (실제 기능 연결은 안 함)

2. 전체 구조 한눈에 보기

역사 사건 자료조사 (위키백과/교과서)
↓
문제별 정답 키워드 정리 + 교차검증 (GPT/Claude/Gemini)
↓
채점 기준 데이터셋 구성 (정답/부분정답/오답 판정 규칙)
↓
화면 디자인 (Framer AI 참고용 시안)
↓

디자인 시안 → HTML/CSS 코드로 변환



음성인식 + 채점 로직(JS) 추가



메모장에 붙여넣고 .html로 저장



크롬으로 열어서 실행 (마이크 권한 허용)

3. Step 1 — 레퍼런스 조사하기

바로 만들기 시작하지 말고, 비슷한 걸 이미 만든 곳들을 좀 보고 시작하는 게 낫다. 참고할 만한 것들:

- 듀오링고 말하기 문제 (음성으로 답하는 인터랙션이 어떻게 생겼는지)
- 발음 평가 앱 (녹음 버튼, 파형 애니메이션 참고)
- 다른 역사 퀴즈 앱 (오답 피드백 방식)

볼 포인트 - 녹음 버튼이 어떻게 생겼고, 녹음 중이라는 걸 어떻게 보여주는지 (색 변화? 파형 애니메이션?)
- 채점 결과를 어떤 방식으로 보여주는지 (즉시? 약간의 딜레이 후?) - 오답이면 바로 정답을 보여주는지, 아니면 힌트만 주는지

캡처든 스케치든 눈에 보이게 몇 개 정리해두면 다음 단계(Framer 디자인)에서 훨씬 수월하다.

4. Step 2 — 문제-정답 키워드 데이터 수집하기

4-1. 다룰 사건 정하기

개항기(1876~1905) 5개 사건으로 범위를 좁혔다.

- 강화도 조약 (1876)
- 임오군란 (1882)
- 갑신정변 (1884)
- 동학농민운동 (1894)

- 을사늑약 (1905)

범위를 넓게 잡으면 문제 하나하나에 신경 쓸 시간이 부족해져서 퀄리티가 떨어진다. 5개 정도로 좁혀야 각 문제의 키워드를 꼼꼼히 검증할 수 있다.

4-2. 위키백과에서 핵심 정보 조사하기

각 사건의 위키백과 문서에 들어가서 연도, 주요 인물, 핵심 결과를 확인한다.

여기서 리뷰 챗봇 만들 때 썼던 **F12 개발자도구 + 콘솔 일괄 추출 기법**을 그대로 다시 쓸 수 있다. 위키백과 페이지에서 F12를 눌러 개발자도구를 열고, 콘솔 탭에 아래처럼 입력하면 본문 텍스트가 한 번에 뽑힌다.

```
document.querySelector('.mw-parser-output').innerText
```

결과가 콘솔에 좌르륵 뜨면 그걸 그대로 복사해서 메모장에 붙여넣으면 된다.

4-3. 조사한 내용 저장하기

메모장 켜서 사건별로 조사한 내용을 정리해 history_raw.txt로 저장한다. 이거면 끝, 스크린샷 같은 거 따로 필요 없다.

[강화도 조약]

1876년, 일본과 체결, 최초의 근대적 조약, 불평등 조약
운요호 사건이 발미, 부산/원산/인천 개항

[임오군란]

1882년, 구식 군인들의 봉기

...

4-4. (자주 묻는 질문) 표로 바로 정리해도 되나

그래도 된다. 오히려 편하다. 텍스트 파일 대신 처음부터 스프레드시트에 사건별로 행을 나눠서 정리해도 상관없다. 4-3은 어디까지나 예시고, 본인이 다음 단계에서 쓰기 편한 형태면 됨.

4-5. 질문 문장 만들기

5개 사건을 각각 자연스러운 질문 문장으로 바꾼다.

1. 강화도 조약이 체결된 연도는?
2. 1876년에 체결된 조약의 이름은?
3. 구식 군인들이 봉기를 일으킨 사건은?
4. 김옥균이 주도한 정변의 이름은?
5. 전봉준이 이끈 농민운동의 이름은?
6. 1905년 대한제국의 외교권을 박탈한 조약은?

사건당 1~2개씩 만들어서 최종 5~10문제 정도로 구성하면 된다.

4-6. AI 톨로 정답 키워드 초안 뽑기

Gemini나 ChatGPT에 4-2에서 조사한 자료를 주고 정답 키워드 초안을 뽑아달라고 요청한다.

아래는 "강화도 조약"에 대해 조사한 자료야.

[여기에 4-2에서 조사한 내용 붙여넣기]

이 사건에 대한 질문에 답할 때, 정답으로 인정하려면 반드시 들어가야 하는 핵심 키워드를 5~7개 뽑아줘.
표 형식으로: 키워드 | 왜 중요한지 한 줄 설명

프롬프트 작성 팁 - 원문 자료를 통째로 붙여넣어야 AI가 근거 있는 키워드를 뽑는다. 요약만 주면 얕은 결과가 나온다 - "키워드 개수를 정해서(5~7개)" 요청해야 결과가 너무 많거나 적어지지 않는다 - "왜 중요한지"를 같이 물어봐야 나중에 필요 없는 키워드를 골라낼 때 판단 기준이 생긴다

5. Step 3 — 정답 키워드 교차검증하기

5-1. 세 모델에 동일한 프롬프트로 키워드 요청

GPT, Claude, Gemini 세 곳에 완전히 똑같은 프롬프트를 넣어서 각각 답을 받는다.

"강화도 조약이 체결된 연도는?"이라는 질문에 대해,
음성으로 답변했을 때 정답으로 인정하려면
꼭 포함되어야 하는 핵심 키워드가 뭐야?
3~5개만 골라서 알려줘.

프롬프트 작성 팁 - 세 모델에 프롬프트를 토씨 하나 안 틀리고 똑같이 넣어야 결과 비교가 의미 있다 -
"3~5개만"처럼 개수 제한을 걸어야 모델마다 결과 개수가 들쭉날쭉해지는 걸 막을 수 있다

5-2. 세 결과를 표로 나란히 정리

문제	GPT 키워드	Claude 키워드	Gemini 키워드	최종(다수결)
강화도 조약 연도는?	강화도, 1876	강화도 조약, 1876년	강화도, 조약	강화도, 1876

5-3. 다수결로 최종 확정

세 모델 결과가 항상 똑같지 않다. 어떤 모델은 "강화도"만 뽑고, 어떤 모델은 "강화도 조약"과 "1876년"까지 자세히 뽑는 식으로 편차가 생긴다. 그래서 한 모델 결과만 믿지 않고, 세 개 중 두 개 이상 겹치는 키워드를 최종으로 채택하는 다수결 방식을 쓴다. 예외 없이 다수결로만 정한다.

6. Step 4 — 채점 기준 데이터셋 구성하기

표 컬럼: 문제 | 정답 키워드(필수) | 정답 판정 기준 | 정답 예시 답변 | 부분정답 예시 답변 | 오답 예시 답변

판정 규칙 - 필수 키워드가 **전부** 답변에 포함되면 → 정답 - 필수 키워드 중 **일부만** 포함되면 → 부분정답 - 필수 키워드가 **하나도** 없으면 → 오답

실전에서 겪은 문제: 통과 기준을 너무 느슨하게 잡으면 생기는 일

처음에는 "키워드 3개 중 1개만 맞아도 통과"로 기준을 잡았었다. 그런데 이렇게 하면 사용자가 문장으로 답 안 하고 **사건 이름 한 단어만 말해도 통과**되는 문제가 생긴다. 예를 들어 정답 키워드가 [강화도조약, 1876, 운요호]인 문제에서, 사용자가 "강화도조약"이라는 단어 하나만 던져도 그게 키워드 중 하나와 일치하니까 통과 처리가 되어버리는 것.

이건 "서술해서 답하기"라는 원래 취지랑 안 맞는다. 그래서 기준을 **"5개 중 3개 이상"**처럼 더 엄격하게 잡아야 한 단어만 말해서는 통과가 안 되고, 실제로 내용을 풀어서 설명해야 통과되게 만들 수 있다. 통과 기준을 정할 때는 꼭 "혹시 키워드 하나만 말해도 뚫리지 않나"를 스스로 점검해보고 정하자.

판정 로직을 실제 코드로 보면 이렇게 생겼다 (8-4에서 AI한테 요청할 기능의 원리)

```
// 정답 키워드 배열과, 그중 몇 개 이상 맞아야 통과인지(통과기준)를 미리 정해둠
const 정답키워드 = ["강화도조약", "1876", "윤요호", "개항", "조약"];
const 통과기준 = 3; // 5개 중 3개 이상

// 인식된 텍스트 안에 각 키워드가 포함됐는지 하나씩 확인해서 개수 셈
let 일치개수 = 0;
정답키워드.forEach(function(키워드) {
  if (인식된텍스트.includes(키워드)) {
    일치개수 = 일치개수 + 1;
  }
});

// 개수에 따라 상태를 정답/부분정답/오답으로 나눔
if (일치개수 === 정답키워드.length) {
  상태 = "정답";
} else if (일치개수 >= 통과기준) {
  상태 = "정답"; // 기준 이상이면 정답 처리
} else if (일치개수 > 0) {
  상태 = "부분정답";
} else {
  상태 = "오답";
}
```

핵심은 딱 이거다: AI가 "이 답이 맞다/아니다"를 스스로 판단하는 게 아니라, 그냥 includes()로 텍스트 안에 단어가 있는지 없는지만 확인하는 단순 비교라는 것. 이게 처음에 짰었던 "규칙 기반 채점" 원칙이 실제 코드에서는 이렇게 구현된다.

6-1. 확정된 키워드 표를 문제별로 정리

5-3에서 다수결로 확정된 키워드를, 위 6개 컬럼 표에 문제별로 하나씩 채워 넣는다.

6-2. AI한테 판정 기준 포함해서 표 완성해달라고 요청

아래는 문제별 정답 키워드 목록이야.

[문제, 키워드 표 붙여넣기]

각 문제마다 아래 컬럼을 채워서 표로 만들어줘.

- 정답 예시 답변 (키워드 전부 포함한 자연스러운 문장)
- 부분정답 예시 답변 (키워드 일부만 포함한 문장)
- 오답 예시 답변 (키워드가 하나도 없는 관련 없는 문장)

프롬프트 작성 팁 - "자연스러운 문장으로"라고 안 하면 키워드만 나열한 부자연스러운 답변이 나온다 - 오답 예시는 "관련 없는 문장"이라고 명확히 지정해야 애매하게 비슷한 문장을 만들어버리는 걸 막을 수 있다

6-3. 부족하면 추가 요청, 표로 저장

문제 수가 부족하거나 예시 답변이 어색하면 추가로 요청해서 다듬는다. 완성되면 스프레드시트에 표로 저장.

6-4. 결과 검토하기

AI가 만든 예시 답변들을 하나씩 눈으로 보면서, 정말 그 판정(정답/부분정답/오답)이 맞는지 직접 검사한다. 여기서 걸러지지 않으면 나중에 앱 테스트할 때 이상한 채점 결과가 나온다.

7. Step 5 — Framer AI로 화면 시안 만들기

7-1. 프롬프트 예시

음성으로 답하는 역사 퀴즈 앱 화면을 만들어줘.
다크 배경에 민트색 포인트 컬러 느낌으로.

포함할 요소:

- 문제 텍스트
- 마이크 녹음 버튼 (대기 상태 / 녹음중 상태로 비주얼이 다름)
- 채점중일 때 뜨는 로딩 표시
- 결과 화면 (정답/부분정답/오답 각각 다른 색 배지)
- 진행 상황 표시 ("5문제 중 2번째")
- 다음 문제로 넘어가는 버튼

프롬프트 작성 팁 - "대기/녹음중 상태로 비주얼이 다름"처럼 상태별 차이를 명시해야 화면이 6개로 제대로 나뉜다 - 색상 톤은 기존 프로젝트 톤(#17171C 배경 + #3FA57A 포인트)을 써도 되고, 이번엔 자유롭게 새로 잡아도 상관없다

7-2. 화면 저장하기

위 상태표 기준으로 화면 6개(대기/녹음중/채점중/정답/부분정답/오답)를 각각 만들고 스크린샷으로 저장해둔다.

7-3. 요소별 영문 이름 붙이기

화면에 있는 요소마다 영문 이름을 붙여둔다. 이 이름이 나중에 코드에서 그대로 id로 쓰인다.

요소	이름
문제 텍스트	QuestionText
녹음 버튼	RecordButton
결과 배지	ResultBadge
진행바	ProgressBar
다음 문제 버튼	NextButton

7-4. 이름표 정리

위 표를 최종본으로 정리해서 다음 단계(코드 생성)에서 그대로 활용한다.

8. Step 6 — ChatGPT로 완성 코드 받기

8-1. 디자인 시안 → HTML/CSS 변환

아래 화면 순서대로 HTML/CSS 코드를 만들어줘.
기능은 아직 넣지 말고 디자인만.
요소 이름은 아래 이름표를 id로 그대로 반영해줘.

화면 순서: 대기 → 녹음중 → 채점중 → 정답 → 부분정답 → 오답

[7-4 이름표 붙여넣기]
[7-2 스크린샷 첨부 또는 설명]

8-2. 화면별로 다듬기

받은 코드를 보면서 색상, 여백, 폰트 크기 등 마음에 안 드는 부분을 화면별로 짚어서 다시 요청한다.

8-3. 엑셀 표를 프롬프트에 넣는 법

방법은 두 가지다.

- **복사-붙여넣기**: 엑셀에서 표를 셀 채로 드래그해서 복사한 다음, 채팅창에 그대로 붙여넣으면 자동으로 표 형태로 인식된다
- **ChatGPT Plus 파일 첨부**: 엑셀 파일 자체를 업로드하는 방법. Plus 요금제에서 가능

둘 다 결과는 비슷하니 편한 쪽으로 쓰면 된다.

8-4. 실제 작동하는 기능 코드 요청하기

위에서 만든 화면에 실제로 작동하는 기능을 추가해줘.

- Web Speech API(webkitSpeechRecognition)를 사용, 언어는 ko-KR로 설정
- RecordButton을 누르면 recognition.start() 실행, 상태를 "녹음중"으로 전환
- 음성 인식 결과 텍스트를 받으면 상태를 "채점중"으로 전환
- 인식된 텍스트와 정답 키워드 목록을 비교해서
포함된 키워드 개수를 세고, 아래 규칙대로 정답/부분정답/오답 상태로 전환
(전부 포함 → 정답, 일부 포함 → 부분정답, 0개 → 오답)
- 마이크 권한 요청 처리 포함 (권한 거부 시 안내 문구도 같이)
- 실제 외부 API 호출(CLOVA 등) 없이, 브라우저 내장 기능과
아래 데이터 안에서만 처리
- html, css, javascript 전부 파일 하나(index.html)에 합쳐서
더블클릭 없이 바로 실행되게
- 요소 id는 아래 이름표 그대로 정확히 연결
- 주석은 한글로

[6절 채점 기준 데이터셋 표 붙여넣기]
[7-4 이름표 붙여넣기]

프롬프트 작성 팁 - "브라우저 내장 API임을 명시"해야 CLOVA 같은 외부 API 코드가 섞여 들어가는 걸 막을 수 있다 - 언어 코드(ko-KR)를 명시하지 않으면 기본적으로 영어로 잡혀서 한국어를 못 알아들을 수 있다

- "파일 하나에 합쳐서"라고 안 하면 html/css/js가 따로 나뉘어서 나오는데, 그럼 다음 단계(메모장 저장)가 복잡해진다 - 판정 규칙(전부/일부/0개)을 프롬프트에 직접 문장으로 써줘야, 표만 던졌을 때보다 로직이 정확하게 짜인다 - "마이크 권한 거부 시 안내 문구"까지 요청해야 실제 테스트할 때 화면이 멍하니 멈춰 있는 상황을 피할 수 있다 - id 이름표를 정확히 붙여서 요청해야 나중에 코드 수정할 때 어디가 뭔지 헷갈리지 않는다

9. Step 7 — 메모장에 저장해서 브라우저로 열기

9-1. 코드 복사

ChatGPT가 준 코드 전체를 복사한다. (html/css/js가 한 파일에 합쳐져 있어야 함)

9-2. 메모장 저장

메모장(또는 다른 텍스트 편집기)을 열어서 붙여넣고, 파일 저장할 때 파일명을 index.html로, 파일 형식을 "모든 파일"로 지정해서 저장한다.

9-3. 브라우저 실행

저장한 index.html 파일을 더블클릭하거나 크롬 브라우저로 드래그해서 연다.

파일을 열면 브라우저가 마이크 권한을 물어보는 팝업을 띄우는데, 여기서 반드시 "**허용**"을 눌러야 녹음 버튼이 작동한다. 거부하면 녹음 버튼을 눌러도 아무 반응이 없다.

10. Step 8 — 테스트하기

5문제(또는 최종 구성한 문제 수)를 실제로 소리 내어 말해보면서 정답/부분정답/오답 케이스를 각각 한 번씩 확인해본다.

발음이 부정확하거나 너무 조용하게 말했을 때 인식이 잘 안 되는 경우도 한 번씩 테스트해보는 게 좋다. 실제 사용 상황에서 흔히 벌어지는 일이라, 미리 확인해두면 나중에 당황하지 않는다.

11. 자주 나는 문제

증상	원인	해결
마이크 권한 팝업이 안 뜨거나 거부됨	브라우저 권한 설정 문제	크롬 주소창 왼쪽 자물쇠 아이콘 → 사이트 설정에서 마이크 허용으로 변경
말을 해도 인식이 안 됨	언어 코드 설정 문제 또는 마이크 미인식	코드에 <code>lang = "ko-KR"</code> 들어갔는지 확인, 다른 마이크로 테스트
정답인데 오답으로 나옴	발음이 다르게 인식되거나 키워드가 너무 뻑뻑함	정답 키워드에 유사 표현 추가 요청 ("강화도조약"뿐 아니라 "강화도 조약"도 인식되게 해줘)
Web Speech API가 아예 작동 안 함	크롬이 아닌 다른 브라우저 사용	크롬 브라우저로 다시 열기 (Web Speech API는 크롬 계열에서 제일 안정적으로 작동함)
녹음은 되는데 텍스트가 안 뜸	네트워크 문제 (Web Speech API는 온라인 상태여야 작동)	인터넷 연결 확인 후 재시도
문제 다음으로 안 넘어감	NextButton 이벤트 연결 누락	ChatGPT한테 "NextButton 클릭 시 다음 문제로 넘어가는 기능이 빠진 것 같다"고 다시 요청

12. 심화: 키워드를 사람 손 대신 알고리즘으로 뽑고 싶다면 (선택)

5절까지의 방법(GPT/Claude/Gemini 세 모델에 물어보고 다수결로 정하기)만으로도 충분히 프로젝트는 완성된다. 그런데 만약 "AI 세 개한테 물어봤다"는 것도 결국 사람이 그 결과를 보고 감으로 고르는 과정이 들어가서, 더 객관적인 방식을 원한다면 아래 방법을 써볼 수 있다. (실제 프로젝트에서 "이 키워드는 어떤 기준으로 고른 거냐"는 피드백을 받고 나서 이 방식으로 바꿨다.)

12-1. 왜 필요한가

GPT한테 "핵심 키워드 뽑아줘"라고 하면 매년 결과가 조금씩 다르게 나올 수 있고, "왜 하필 이 단어를 골랐는지"에 대한 설명이 부족하다. 통계적으로 "이 단어가 실제로 이 사건 설명 안에서 얼마나 중요한 위치를 차지하는지"를 계산해서 뽑으면, 더 재현 가능하고 설명하기 쉬운 근거가 생긴다.

12-2. 처리 과정 요약

사건별 설명 원문 텍스트 준비

↓

형태소 분석으로 명사만 추출 (조사, 어미 등 제거)

↓

TextRank 알고리즘으로 단어 중요도 순위 매기기

↓

공통 일반어(여러 문제에 다 나오는 단어) 점수 낮추기 - IDF 재가중

↓

연도·인물명은 규칙으로 우선 포함

↓

최종 키워드 5개 확정

12-3. 형태소 분석이란

문장을 의미 있는 최소 단위(형태소)로 쪼개고, 그중 명사만 남기는 작업이다. "강화도 조약이 체결되었다"를 분석하면 "강화도 / 조약 / 체결" 같은 명사만 남고 조사나 어미는 버려진다. 파이썬에서는 kiwipiepy 같은 한국어 형태소 분석 라이브러리를 쓰면 된다.

```
pip install kiwipiepy
```

12-4. TextRank란

구글 페이지랭크랑 비슷한 원리로, 문서 안에서 다른 단어들과 자주 같이 등장하는 단어일수록 점수를 높게 매기는 알고리즘이다. 사람이 감으로 "이게 중요해 보인다"고 고르는 대신, 통계적으로 계산해서 순위를 매긴다는 게 핵심이다.

12-5. IDF 재가중이 필요한 이유

TextRank만 돌리면 "조선", "일본"처럼 여러 문제에 공통으로 자주 나오는 일반적인 단어가 오히려 상위에 뽑히는 문제가 생긴다. 이걸 막기 위해 IDF(역문서빈도)라는 값을 곱해서, 여러 문제에 공통으로 등장하는 단어는 점수를 낮추고 그 문제에서만 특징적으로 등장하는 단어는 점수를 높인다.

공식

$$IDF = \log(\text{전체 문항 수 } N / \text{그 단어가 등장한 문항 수 } df)$$

예를 들어 5문항 중 3문항에 나오는 단어면 $\log(5/3)$, 1문항에만 나오는 단어면 $\log(5/1)$ 로 값이 더 커진다. 즉 여러 문제에 공통으로 등장하는 단어일수록 IDF 값이 낮아지고, 특정 문제에서만 튀는 단어일수록 IDF 값이 높아진다.

최종 점수는 TextRank 점수 $\times$ IDF로 계산해서 다시 순위를 매긴다.

12-6. AI 툴로 이 코드 짜는 법

형태소 분석, TextRank 계산, IDF 재가중까지 전부 직접 코드로 짜기는 번거로우니, ChatGPT나 Claude한테 아래처럼 요청하면 된다.

파이썬으로 다음 파이프라인을 구현해줘:

1. kiwipiepy로 한국어 텍스트에서 명사만 추출
2. 추출된 명사로 TextRank 그래프를 만들어서 단어 중요도 순위 매기기 (networkx의 pagerank 함수 활용)
3. 문제 5개 각각에 대해 계산한 뒤, 여러 문제에 공통으로 등장하는 단어는 IDF로 점수를 낮추는 재가중 적용
4. 최종적으로 문제별 상위 5개 키워드를 출력

입력 데이터는 아래와 같아:

[문제별 원문 텍스트 붙여넣기]

프롬프트 작성 팁 - 사용할 라이브러리 이름(kiwipiepy, networkx)을 미리 지정해야 엉뚱한 라이브러리로 짜는 걸 막을 수 있다 - "문제 5개 각각에 대해"라고 명시해야 전체를 뭉뚱그려서 한 번에 계산하는 실수를 막을 수 있다 - IDF 공식을 그냥 "가중치 적용해줘"라고만 하면 다르게 짤 수 있으니, 위 12-5의 공식을 그대로 프롬프트에 넣어주는 게 안전하다

12-7. 결과 검토는 여전히 필요하다

알고리즘으로 뽑았다고 끝이 아니다. 결과를 보면 여전히 이상한 단어가 상위에 남아있을 수 있다 (예: 지명이 인명으로 잘못 분류되는 경우). 이럴 땐 연도·핵심인물처럼 학습상 꼭 필요한 항목은 규칙으로 강제 포함시키고, 나머지 슬롯만 알고리즘 결과로 채우는 식으로 사람이 마지막에 한 번 더 점검하는 게 좋다. 완전 자동화보다 "알고리즘 + 최소한의 사람 검토"가 실제로는 더 안정적이다.

13. 마무리 체크리스트

- [] 개항기 5개 사건 자료조사 마쳤다
- [] GPT/Claude/Gemini 3모델 교차검증 + 다수결로 정답 키워드 확정했다
- [] 채점 기준(문제/키워드/판정규칙) 데이터 정리했다
- [] Framer로 화면 시안 6개 만들었다
- [] 화면(HTML/CSS)과 기능(JS, 음성인식 포함)을 나눠서 각각 받았다
- [] 메모장 → .html 저장 → 크롬 실행까지 확인했다
- [] 마이크 권한 허용하고 정답/부분정답/오답 다 테스트해봤다